

AULA 5 - INTRODUÇÃO E REPRESENTAÇÃO

Grafos são estruturas de dados usadas para representar elementos e suas interligações; é uma ferramenta de modelagem que permite analisar e resolver problemas.

- encontrar o caminho mais curto entre dois pontos
- comunicação
- algoritmos de busca

O problema das sete pontes de Königsberg

Atravessar todas as pontes uma vez e retornar ao ponto de partida.

terra: vértices; pontes: arestas.

Os vértices deviam ter um número par de arestas (sempre que chegar por um vértice você pode sair dele por outra aresta, basicamente tem uma aresta que sai e uma que entra).

grafo conexo: deve ser possível alcançar qualquer vértice a partir de qualquer outro vértice.

a cidade não atendia a condição de aresta par.

Definições:

Seja $G(V, E)$ um grafo, V é o conjunto não vazio de vértices, e E o conjunto de arestas.

Cada aresta é um par ordenado (u, v) , [partida e chegada respectivamente].

- incidência: relação entre os vértices e arestas de um grafo
- adjacência: relação de conexão entre dois vértices por uma aresta; Dois vértices são adjacentes se eles estão diretamente conectados por uma aresta. É como dizer que são "vizinhos" no grafo; o tanto de arestas que "saem" do vértice é sua adjacência.
- dígrafo: grafo dirigido/direcionado
 - $e1 = (a, b) \neq (b, a)$
 - $e1 = (a, b) = (b, a)$
- grafo misto: grafo com arestas orientadas e não-orientadas
- laço: uma aresta que conecta um vértice a ele mesmo
- arestas paralelas: duas ou mais áreas que conectam o mesmo par de vértices
- grafo simples: não tem arestas paralelas e nem laços
- multigrafo: tem pelo menos um conjunto de arestas paralelas ou um laço
- grafo nulo: sem arestas

Parâmetros

- ordem: número de vértices, $|V|$
- tamanho: número de arestas, $|E|$
- densidade: relação entre a ordem e o tamanho (grafo esparso - mais vértices do que arestas; grafo denso - mais arestas do que vértices).

percurso - total de arestas percorridas

grau do vértice:

- se direcionado: arestas que entram + arestas que saem
- não direcionado: número de arestas que incidem no vértice (saem do vértice)

Formas de representação de grafos

- Definição de conjuntos, $G = (V, E)$
- Gráfica
- Listas de arestas - lista não ordenada de arestas; é uma implementação simples mas a operação de busca de arestas não é eficiente, e a localização de todas as arestas incidentes a um vértice também não é eficiente. Implementado com listas.

Na representação por lista de arestas:

- Cada aresta é armazenada como um par (ou tupla) de vértices.
- Se o grafo for **não direcionado**, as arestas são pares não ordenados, por exemplo, $\{u,v\}$.
- Se o grafo for **direcionado**, as arestas são pares ordenados, por exemplo, (u,v) onde u é o vértice inicial e v o vértice final.
- Listas de adjacência
 - cada vértice mantém uma lista não ordenada de todas as suas arestas incidentes; Isto permite que a determinação das arestas incidentes a um vértice seja mais eficiente (dado um vértice já temos a lista de suas adjacências); Implementado com lista(s).
 - no caso da matriz temos que varrer a linha correspondente a esse nó, buscando as arestas.
- Mapas de adjacência
 - Semelhante à listas de adjacência, mas ao invés de uma lista, cada vértice mantém um mapa de arestas.
 - As chaves do mapa são os vértices adjacentes ao vértice considerado.
 - Permite uma maior eficiência ao acesso a uma determinada aresta.
 - Implementado com hashmaps: o vértice incidente oposto de cada aresta é a chave do par chave/valor do mapa.
 - Vantagem sobre as listas de adjacência é que a operação de busca de uma aresta com base em seus vértices ocorre em tempo constante $O(1)$.
- Matrizes de adjacência
 - Matriz $n \times n$.
 - Preenchida com zero nas posições em que o elemento da linha não tem ligação com o elemento da coluna.
 - Preenchida com um nas posições em que o elemento da linha tem ligação com o elemento da coluna, ou seja, quando há uma aresta.
 - Estrutura fácil de implementar.
 - Acesso a arestas em tempo constante.
 - Em grafos não dirigidos, a matriz é simétrica.

As duas primeiras são úteis para estudo teórico de grafos, ao passo que as quatro últimas são mais adequadas para implementação computacional.

Em suma:

Matriz de adjacência para: grafos densos, operações como teste de arestas e identificação de predecessores.

Lista de adjacência para: grafos esparsos, operações que tenham como base um caminho de um vértice a outro (ex: busca).

com grafos mais densos (mais aresta que vértice) a lista ocupa mais memória

v1 -> v2 -> v3 -> v4

v2 -> v1 -> v3 -> v4 -> v5

AULA 6 - BUSCA E CONEXIDADE

Fazer buscas em um grafo significa seguir suas arestas sistematicamente de modo a visitar seus vértices.

BUSCA EM PROFUNDIDADE (DFS)

Começa de um nó e vai o mais fundo possível antes de retroceder.

Paramos quando encontramos o que procuramos ou quando visitamos todos os vértices e não encontramos.

As arestas são exploradas a partir do vértices v mais recentemente descoberto que ainda tem arestas não exploradas dele

Quando todas as arestas de v são exploradas a busca volta ao vértice anterior a v (backtracking) para seguir arestas ainda não exploradas.

Pseudocódigo da Busca em Profundidade (DFS)

1. Inicialize todos os vértices como não visitados.
2. Comece no vértice inicial e, recursivamente, visite todos os seus vizinhos ainda não visitados.
3. Marque o vértice como encerrado quando todos os seus vizinhos tiverem sido explorados.

*não garante o caminho mais curto.

**Usa lista de adjacência.

BUSCA EM LARGURA (BFS)

Primeiro visita todos os adjacentes de um nó, para então visitar os adjacentes dos adjacentes; "em camadas".

dado um nó inicial S , a busca determina a distância (n arestas) de cada vértice atingível do grafo a S .

*menor distância em número de arestas!

Pseudocódigo da Busca em Largura (BFS)

1. Inicialize todos os vértices como não visitados.
2. Insira o vértice inicial na fila e marque-o como visitado.
3. Enquanto a fila não estiver vazia, faça:
 - Remova o vértice da frente da fila.
 - Para cada vizinho não visitado do vértice atual, insira-o na fila e marque-o como visitado.
 - Atualize o vetor de predecessores para indicar de onde o vértice foi alcançado.

CONEXIDADE

A conexidade de um grafo trata das partes dele estarem ou não conectadas; os grafos podem ser desconectados em componentes menores chamados componentes conexos.

Um grafo é conexo se para todos os pares de vértices existe um caminho que os conecta.

grafo subjacente - transformação de um grafo dirigido em não dirigido ao remover a direção das arestas.

Conexidade em grafos dirigidos

Grafo Fortemente Conexo: Um grafo dirigido é fortemente conexo se existe um caminho entre todos os pares de vértices, em ambas as direções.

AULA 7 - CICLOS E ÁRVORES

GRAFOS EULERIANOS - Referem-se ao problema das pontes de Königsberg; um grafo é euleriano se contém uma trilha fechada que visita cada aresta uma vez, formando um Ciclo Euleriano.

GRAFOS HAMILTONIANOS - inspirado em um jogo cujo objetivo é passar por todos os vértices uma vez retornando ao ponto de partida, formando um Ciclo Hamiltoniano.

Grafos e Ciclos Eulerianos

Definições e Propriedades:

Ciclo Euleriano: Um ciclo que passa por cada aresta do grafo exatamente uma vez e retorna ao ponto de partida.

Teorema de Euler: Um grafo é Euleriano se, e somente se, todos os seus vértices tiverem grau par.

Grafos Semi-Eulerianos: Possuem exatamente dois vértices de grau ímpar. Nesses grafos, existe um caminho semi-euleriano, ou seja, um caminho que passa por todas as arestas sem formar um ciclo fechado.

Grafos e Ciclos Hamiltonianos

Definições e Propriedades:

Ciclo Hamiltoniano: Um ciclo que passa por todos os vértices exatamente uma vez e retorna ao ponto de partida.

Teorema de Ore: Se, em um grafo de n vértices, qualquer par de vértices não adjacentes u e v satisfaz $\text{grau}(u) + \text{grau}(v) \geq n$, então o grafo é Hamiltoniano.

Diferença para Euleriano: Ao contrário dos grafos Eulerianos, não há uma condição fácil para determinar se um grafo é Hamiltoniano. Isso torna o problema mais desafiador e, em muitos casos, requer tentativas ou aproximações.

Conclusão

- Ciclos Eulerianos podem ser detectados facilmente e possuem soluções eficientes, como o Algoritmo de Hierholzer.
- Ciclos Hamiltonianos são mais desafiadores e exigem abordagens como backtracking ou aproximações, mas têm relevância em problemas práticos, como o Caixeiro Viajante.

ÁRVORES

- Árvores: Estruturas hierárquicas que não possuem ciclos e conectam todos os vértices. Uma árvore com n vértices tem exatamente $n-1$ arestas.
- Árvores Geradoras: subconjuntos de um grafo que incluem todos os vértices, mas sem ciclos, e têm o número mínimo de arestas ($n-1$ arestas).
- Grafos Valorados: as arestas possuem valores

- Árvores Geradoras de Custo Mínimo: Entre todas as possíveis árvores geradoras de um grafo valorado, a árvore geradora de custo mínimo é aquela cuja soma dos pesos das arestas é mínima (a menor possível).

Algoritmos de Árvores Geradoras de Custo Mínimo

- Algoritmo de Prim
- Algoritmo de Kruskal

Aplicações Reais:

- Redes de Computadores: As árvores geradoras de custo mínimo são usadas para minimizar o custo de construção de redes de comunicação, como a internet.
- Sistemas de Distribuição de Energia: Utilizam árvores geradoras de custo mínimo para otimizar a distribuição de energia elétrica.
- Planejamento de Rotas: Utilizam árvores para encontrar o caminho mais curto entre várias localidades.

Conclusão:

- Árvores geradoras de custo mínimo são fundamentais em diversas áreas, com algoritmos eficientes como Prim e Kruskal.
- Grafos valorados permitem a modelagem de problemas reais, onde o objetivo é minimizar custos ou distâncias.

AULA 8 - CAMINHO MÍNIMO

O problema do caminho mínimo em grafos valorados consiste em encontrar um caminho com custo total mínimo entre um vértice inicial e um vértice final.

Este tipo de resultado pode ser obtido tanto em grafos dirigidos quanto em grafos não dirigidos, e os algoritmos para isso podem ser aplicados em ambos os tipos de grafos.

O problema de caminhos mínimos em grafos possui diversas aplicações em áreas como redes, rotas e logística.

Custo de um caminho entre u e v : é a soma de pesos das arestas que conectam u a v .

Por exemplo, se $u = 1$ e $v = 4$, e tem-se $u \rightarrow a$ com peso 4 e $a \rightarrow v$ com peso 5, o custo desse caminho é 9.

Custo do caminho mínimo: o custo do menor caminho que conecta u a v . Pegando o mesmo exemplo acima, se houver $u \rightarrow a$ com peso 4, $a \rightarrow v$ com peso 5 e $u \rightarrow v$ com peso 8, o custo do caminho mínimo entre u e v é 8. Se não existir um caminho conectando u a v , diz-se que o custo é infinito.

Problema do Caminho Mínimo: Consiste em encontrar o caminho de menor custo entre dois vértices de um grafo, onde cada aresta possui um peso (custo).

Para resolver o problema do caminho mínimo, são apresentados dois algoritmos:

- Dijkstra
- Floyd-Warshall

Algoritmo de Dijkstra

- Funciona para grafos com pesos não negativos e calcula o menor caminho a partir de um vértice inicial para todos os outros. Constroi uma árvore de caminhos mínimos, expandindo-se a partir do vértice inicial.
- Possui complexidade $O(E + V \log V)$ se implementado com fila de prioridade e $O(V^2)$ se implementado com array.

Algoritmo de Floyd-Warshall

- Também chamado de Algoritmo de Floyd, Roy, Roy-Warshall e outras variações.
- Calcula o caminho mínimo entre todos os pares de vértices em um grafo e é adequado para grafos com pesos negativos (mas sem ciclos negativos). Ele utiliza uma matriz de distâncias, que é atualizada em etapas para refletir os menores custos entre cada par de vértices.
- Possui complexidade $O(V^3)$, onde V é o número de vértices.